

Consolidated services server — minimum specification

The server itself is already on hand (see docs/topology.md) — this document calculates the **target minimum spec** to check that hardware against, working forward from the actual workload this box runs (docs/topology.md 's VM/container table) rather than a generic "decent server" guess. Where a figure depends on something this repo hasn't pinned down (the event's actual day count/hours), that's flagged explicitly rather than assumed — see docs/open-questions.md for the running list.

Storage — three pools, not one array

Unraid's traditional parity array is built around mixed-size spinning disks with a single-write-at-a-time parity bottleneck, absorbed by an SSD cache pool that the overnight "mover" drains onto the array — a good fit for bulk archival storage, a poor fit for a box whose entire storage workload here is already flash. Recommendation: skip the legacy array entirely and use three separate Unraid **pools** (ZFS, available since Unraid 6.12), each sized and mirrored for what it actually holds.

Container/VM pool

Holds docker.img , all container appdata (including the three databases — MariaDB, MongoDB, Redis — that want fast, low-latency small-I/O access), and the BirdDog Central VM's vdisk.

Item	Estimate
BirdDog Central vdisk	~80 GB (OS + app)
docker.img	~50 GB
Combined appdata (12 containers + the Tailscale subnet router + 3 DBs)	~58 GB
Total	~188 GB

Minimum: 500 GB usable, mirrored NVMe (2× ~512GB+ NVMe) — comfortable headroom over the ~188 GB estimate, and mirrored because rebuilding a Windows VM from scratch mid-event is real downtime, not just an inconvenience. NVMe specifically (not SATA SSD) because VM and database performance is latency-sensitive, not throughput-bound.

Drive class: mainstream consumer NVMe with onboard DRAM cache (not a DRAM-less budget drive) is genuinely sufficient here — no need for the enterprise tier the recording pool below needs. VM/appdata churn is nowhere near enough sustained write volume to meaningfully test even consumer-grade endurance ratings, and the workload cares about consistent low-queue-depth latency (how snappy a container restart or a DB query feels), which good consumer NVMe already delivers.

Content pool — 500 GB (as specified)

Nextcloud's own storage: theatre laptop content synced via rclone (established at 10-20 GB/theatre × 12 theatres = **120-240 GB**, see docs/bandwidth-analysis.md), plus Nextcloud's own app/database footprint and versioning overhead (flagged as a real multiplier in docs/atem-iso-ingest.md 's versioning caveat — version retention settings directly affect how much of this budget versioning eats into). 500 GB gives roughly 2-4x headroom over the raw 120-240 GB content estimate, which comfortably absorbs version history without needing an aggressive retention policy from day one.

Recommendation: mirrored, SATA SSD is adequate (no NVMe-level latency requirement here) — mirrored less for data-loss prevention (every laptop is still the source of truth for its own content even if this copy is lost) and more for **availability**: a lost content pool mid-event means re-running the initial bulk sync for all 12 theatres, which is itself only a ~20-40 minute operation at 50-100 Mbps (docs/bandwidth-analysis.md) but is a real, avoidable disruption during a live event.

Drive class: same reasoning as the container/VM pool — mainstream consumer SATA SSD with DRAM cache, the cheapest reasonable tier that still isn't a DRAM-less budget part. Gentle, sporadic write pattern; no case for spending more here.

Recording pool — 8 TB, all-flash (as specified)

This is where the calculation actually matters — both *why* it has to be flash and *whether 8 TB is enough*.

Why all-flash, not spinning disk — it's not about raw throughput. Combined worst-case write load onto this pool, from docs/bandwidth-analysis.md 's own established figures:

Source	Per-unit high case	Count	Total
ATEM ISO ingest, default live scope (camera ISO + program)	~80 Mbps (~83 Mbps incl. WireGuard tax)	12 theatres	~960 Mbps
VMix Record ingest	~30 Mbps (assumed, unconfirmed — see docs/open-questions.md)	4 PCs	~120 Mbps
Combined			~1,080 Mbps ≈ 135 MB/s
If both laptop inputs are also pulled live (ATEM_ISO_INPUTS=1,2,3)	~160 Mbps	12 theatres	~2,040 Mbps ≈ 255 MB/s combined

(ATEM figures are the 2026-10-06 planning figures — camera ISO 35 / 45 / 70 Mbps, laptop ISO 15 / 25 / 40 Mbps *estimated*, program ~10 Mbps — for the program file plus only the ISO inputs pulled live; see docs/atem-iso-ingest.md § Common theatre input map. The pool stores payload, so WireGuard overhead isn't counted.)

135-255 MB/s sustained is within a single 7200 RPM HDD's *rated sequential* throughput on paper — so the case for flash isn't the aggregate number, it's the **access pattern**. That load isn't one stream, it's **~28 independent files being appended to concurrently** (12 theatres × camera ISO + program, plus 4 VMix — ~52 with both laptop inputs live), each on its own ~60-90s poll cycle — see docs/atem-iso-ingest.md and docs/vmix-record-ingest.md), with Nextcloud's own targeted `occ files:scan` indexing calls layered on top of that. A spinning disk pays a real seek penalty (~5-15ms) every time it switches between those ~28+ scattered write targets — under this specific concurrent-scattered pattern, achievable HDD throughput collapses to a small fraction of its rated sequential number, while any flash drive (no seek

penalty) handles 135-255 MB/s of concurrent random-ish writes without strain. The "all-flash" requirement is about the write *pattern* this design creates, not the volume.

Does 8 TB actually cover the event? This is the one figure genuinely blocked on an input this repo doesn't have: **actual event duration (days × active-recording hours/day) isn't established anywhere in this design.** Working from the same combined-load figures:

Live scope (all 12 theatres)	Combined rate, typical / high	8 TB covers
Camera + program (default)	~98 / 135 MB/s ≈ 351 / 486 GB/hr	~23 / ~16.5 hours
Camera + 1 laptop + program	~135 / 195 MB/s ≈ 486 / 702 GB/hr	~16.5 / ~11.4 hours
Camera + 2 laptops + program	~173 / 255 MB/s ≈ 621 / 918 GB/hr	~12.9 / ~8.7 hours

(Decimal units throughout — 8 TB = 8,000 GB, as drives are sold. Rate = 12 × the per-theatre ATEM payload (camera + program: 55 typical / 80 high Mbps; each laptop adds 25 / 40) + the same ~120 Mbps VMix placeholder, ÷ 8. Low-case figures (camera ISO at ~35 Mbps) stretch the default-scope coverage to ~27 hours. All figures are to a *completely* full pool; ZFS performance degrades well before 100%, so treat roughly 85-90% of each as the practical figure.)

Basis — planning figures, not measurements. The old ~10 Mbps-per-ISO basis was replaced on 2026-10-06 (Blackmagic: ISO files up to 70 Mb/s, not user-settable); the table above instead assumes only the program file and the chosen ISO inputs are pulled live, at the per-source planning figures in `docs/atem-iso-ingest.md` § Bandwidth. The camera figure rests on user reports and the laptop figure is an unmeasured estimate — one measured file of each settles it (`docs/open-questions.md` #0). The ISOs *not* pulled live arrive via the physical DIT offload to the edit-suite NAS (`docs/live-editing.md`), not this pool; if they are also to be archived here, add each ATEM's full recording (at least ~14-36 GB per theatre-hour more for the two laptop ISOs) to the rate.

At a typical ~8-10 hour active-program day, that's roughly **1.6-2.9 days** of recording at the default scope (high to typical rates) before the pool fills — workable for a short event, tight for a longer one — and only ****~0.9-1.6 days**** if both laptop inputs are pulled live fleet-wide. **Confirm actual event length against this table before treating 8 TB as settled** — if it's a multi-day event without a periodic archive-off step already planned, either the day count needs to fit the budget above, or a nightly archive-to-cold-storage step needs adding to free capacity for the next day. The ingest pipelines write directly into Nextcloud's External Storage location and, in the same pull, the edit-suite NAS (`docs/atem-iso-ingest.md` , `docs/live-editing.md`) — neither write has an offload stage that frees *this* pool. Tracked in `docs/open-questions.md` .

Redundancy — worth adding capacity for, not assumed in the 8 TB figure. This pool is the authoritative archive: the dual-write to the edit-suite NAS (`docs/live-editing.md`) does give the footage a second landing spot at ingest time, but that copy may be a curated subset (open-questions #12) and one of the candidate NAS units is RAID 0 — so a failure here still risks footage that can't be re-shot. Two ways to get 8 TB *usable* with 1-drive fault tolerance:

Option	Raw capacity needed	Trade-off
2× 8 TB NVMe, mirrored	16 TB raw	Simplest to reason about, fastest resilver, 50% capacity overhead
4× ~2.7 TB NVMe, RAID-Z1	~10.8 TB raw	75% capacity efficiency vs. 50%, slower resilver on failure, more drives to manage

Real drive sizes don't land on these numbers. Enterprise NVMe in the ~1 DDPD tier is sold in 1.92 / 3.84 / 7.68 / 15.36 TB steps (e.g. Samsung PM9A3), so "2× 8 TB mirrored" is in practice **2× 7.68 TB ≈ 7.7 TB usable** (a few percent under 8 TB before ZFS's own reserve), and there is no ~2.7 TB SKU — the realistic RAID-Z1 build is **4× 3.84 TB ≈ 11.5 TB usable** (15.36 TB raw). If 8 TB usable is a hard floor, the mirror needs 2× 15.36 TB; otherwise 2× 7.68 TB is the honest reading of "8 TB" and the coverage table above shrinks by ~4%.

Recommend the mirror for the same reason Unraid was chosen over TrueNAS SCALE elsewhere in this design (docs/topology.md) — operational simplicity, since this may be maintained by AV staff rather than a storage specialist, matters more here than squeezing out the last few TB of efficiency.

Endurance. Since this repo already treats the router hardware as reusable across events (see docs/gl-inet-rationale.md), the same applies here — a full pool fill is roughly 8 TB of actual flash writes (these are byte-range *appends*, not whole-file rewrites, so total written ≈ total recorded, not a multiple of it), and a mirrored pair each independently absorb that same 8 TB per event. Even a deliberately-generous worst-case estimate — ~920 GB/hr sustained (both laptop inputs live, high rates) for a full 24 h/day across ~20 event-days a year, beyond what the pool could even hold without nightly archive-off — comes to roughly 440 TB/year of actual writes to the pool — comfortably inside what even the *lower* enterprise endurance tier (see below) is rated for over a normal warranty period, so endurance headroom isn't actually the tight constraint here once the right drive class is picked.

Drive class: this is the one pool where consumer-grade flash — even a good, DRAM-cached, high-TBW consumer drive — isn't the right call, and endurance isn't the reason. The deciding factor is **power-loss protection (PLP)**: capacitor-backed circuitry that flushes a drive's write cache to permanent storage if power drops mid-write. Consumer/prosumer NVMe drives don't have this. A generator hiccup, a tripped breaker, or a UPS transfer glitch mid-event — exactly the kind of thing a live event's own power setup can produce — risks losing whatever was sitting in the drive's write cache at that instant, on the one pool in this whole design with no second copy of the data anywhere else. That's the actual argument for **enterprise-class NVMe** here specifically (not "enterprise is always better," which isn't true for the other two pools above) — server-validated drives with capacitor-backed PLP, provisioned for sustained high-queue-depth writes without the cache-cliff behavior consumer drives show once their fast write cache fills under prolonged concurrent load. Enterprise drives are also sold in more than one endurance tier (commonly around 1 drive-write-per-day vs. 3× that for a heavier-duty tier) — the lighter of the two tiers already has large margin over this workload's realistic annual write volume, so there's no need to pay for the heavier tier's endurance on top of the PLP requirement that's actually driving this choice.

Write caching

With every pool already flash (no legacy spinning array behind anything), the classic Unraid "SSD cache pool absorbing writes before the overnight mover" pattern doesn't apply here — there's no slow array for it to shield. What actually buffers the bursty, ~28-concurrent-stream ingest pattern described above is:

- **RAM** — the OS page cache smooths write bursts before they hit the drives (see the RAM section below, which already budgets for this).
- **The drives' own onboard DRAM cache** — a real purchasing consideration: choose NVMe drives with onboard DRAM (not DRAM-less/QLC-only budget drives), since sustained multi-stream concurrent writes are exactly the workload DRAM-less drives handle worst.

NIC — validating the already-decided 2× bonded GbE

`docs/topology.md` already settled on 2× gigabit NICs, bonded (802.3ad LACP). Checking that against the heaviest combined load this design has identified — the mass-NDI-fallback disaster scenario from `docs/bandwidth-analysis.md`, layered on top of full ATEM ingest:

Direction	Load	Total
Inbound (theatres → mothership), default live scope (camera + program)	ATEM ingest incl. WireGuard (~686 typical / ~998 high Mbps) + VMix ingest (~120 Mbps) + ATEM Overseer (~125 Mbps) + Flock (~125 Mbps) + rclone (~60 Mbps)	~1,120 typical / ~1,430 high Mbps
Inbound, camera + 2 laptops + program everywhere	ATEM ingest ~1,310 typical / ~1,997 high Mbps + the same ~430 Mbps	~1,740 typical / ~2,430 high Mbps
Outbound (mothership → theatres), all 12 theatres on NDI fallback simultaneously	12 × ~130 Mbps	~1,560 Mbps

(Recomputed 2026-10-06 from the per-source planning figures in `docs/atem-iso-ingest.md` § Bandwidth, replacing the earlier ~1,493 Mbps that rested on ~10 Mbps per ISO stream; rclone's ~5 Mbps per theatre is now counted too. Matches `docs/bandwidth-analysis.md`.)

Both directions run independently over a full-duplex link, so they don't stack against each other — but each direction needs to fit across the bond's two physical 1 Gbps links via LACP's per-flow hashing (each *individual* flow is capped at 1 Gbps, since LACP doesn't split one flow across both links).

How many flows the hash actually sees is the catch. Everything to and from the theatres crosses the wire as **WireGuard**, and the bond hashes the *outer* packet — so each theatre's ATEM ingest + Overseer + Flock + rclone (and, outbound, its SRT or NDI) collapse into **one UDP flow per tailnet peer**: ~14 flows in each direction, not 36+. Each is still far under 1 Gbps (~109 Mbps per theatre inbound at the default scope's high case, ~130 Mbps outbound during NDI fallback), so no *single* flow is the problem — but 12-14 roughly equal flows split 2 ways by an effectively random hash are often uneven. Worked out exhaustively for the high cases above (scratch calculation over all 2¹⁴ assignments, treating each flow's link as a coin flip): ****~21% of possible hash assignments put more than ~940 Mbps of usable payload on one link inbound at the default scope's high rates, and ~39% for the NDI outbound case**** (8 of 12 NDI flows × 130 Mbps = 1,040 Mbps). With both laptop inputs live fleet-wide it is ~77% inbound even at typical rates — effectively "doesn't fit" on 2× 1GbE, and the high case (~2,430 Mbps) exceeds the bond's aggregate outright. Real hashes are deterministic per peer address/port, so the result is "fine or not, for the whole event," not random per minute — and can't be predicted on paper.

So, at the default live scope: **the aggregate fits (~1,430 Mbps inbound at high rates, ~1,560 outbound, vs. ~2,000), and normal operation (~1,120 Mbps typical inbound, no NDI fallback — ~1% of hash assignments overload a link) is comfortable — but the worst-case disaster scenario is only "probably fits," not "checks out,"** on 2× 1GbE — and that is *before* the mothership-side routing hairpin described below, which on its own breaks the NDI-fallback case unless the VM is routed directly. It does check out once ATEM ingest is paused fleet-wide during a mass NDI fallback (the mitigation below), and it checks out unconditionally on 2× 2.5GbE (see below). Pulling the laptop inputs live fleet-wide does *not* fit 2× 1GbE (see above) — it needs 2× 2.5GbE, or the laptop inputs enabled for only some theatres (`ATEM_ISO_INPUTS` is currently one fleet-wide setting). The inbound figure has moved each time this was revalidated (~1,205 → ~1,493 → now ~1,430 Mbps at the default scope's high case) as Overseer, Flock and the ISO bitrate correction landed — see `docs/bandwidth-analysis.md` 's own worked-through

comparison of this exact disaster scenario with and without pausing ATEM ingest fleet-wide, which is the more consequential mitigation than the NIC choice itself.

One load deliberately NOT counted against this bond: the dual-write to the edit-suite NAS. The same ingest containers that produce the inbound figures above also write every recording out again to the edit-suite NAS at 192.168.22.2 (docs/live-editing.md) — another ~1,080 Mbps of outbound at the default scope's high case, up to ~2,040 Mbps with both laptop inputs live. If that leg transited the theatre-facing bond it would stack on top of the NDI-fallback outbound and break the conclusion above — so it must **not**: the edit suite is its own 10GbE LAN, and this box needs a separate edit-LAN-facing interface (a 10GbE NIC, or at minimum its own dedicated port) for that traffic, counted in the spec alongside the 2× bonded theatre-facing GbE.

Worth it: 2× 2.5GbE, if the box and whatever terminates the bond support it. This is the one change that removes the hash-imbalance risk above outright at the default scope — even the worst possible split (all ~1.4-1.6 Gbps on one link) fits inside a single 2.5 Gbps link — and it is what makes pulling both laptop inputs live fleet-wide viable (~2.4 Gbps worst split at high rates: just inside one link, no headroom). Many current motherboards ship 2.5GbE onboard already; the constraint is more likely the far end of the bond than the server.

Two things upstream of the bond can cap this before the NIC does — neither is about the server, both are worth checking before trusting any figure in this section:

- **The Cloud Gateway's own routing capacity.** Every theatre packet arrives on an uplink VLAN and is *routed* onto Mothership-LAN by the gateway. On a UDM Pro, the 8-port LAN switch is reported to reach the CPU over a single 1 Gbps link on current hardware revisions, capping inter-VLAN routing at ~1 Gbps regardless of LAG (community wiki — not a Ubiquiti spec sheet, verify against the actual unit). The exact model is still unconfirmed — docs/open-questions.md #17.
- **A routing hairpin on the mothership side.** The mothership's containers and the BirdDog Central VM reach the theatre subnets via the Tailscale subnet router at 192.168.1.2 . That path is now implemented as static routes on the Cloud Gateway (config/unifi/network-config.yaml) — the macvlan containers can't use the host as a next hop directly, and the VM uses the gateway as its default route — so every packet crosses the bond twice (container/VM → gateway → back to the host's tailscaled , then out again as WireGuard). For the BirdDog Central NDI fallback that makes the server's transmit ~3,100 Mbps (VM → gateway, then host → theatres), not ~1,560 — over even the bond's aggregate. Mitigations: give the VM its own static routes to the theatre /24 s via 192.168.1.2 (a VM on Unraid's bridge *can* reach the host directly, unlike a macvlan container), and/or 2× 2.5GbE. The return leg and the ACL side are still open — docs/open-questions.md #18.

RAM

Workload	Estimate	Basis
BirdDog Central VM	8 GB	Routing/control app, not encode
Nextcloud + MariaDB + Redis	6 GB	InnoDB buffer pool + PHP workers + Redis cache
UniFi Controllor + MongoDB	3 GB	Manages only the Cloud Gateway (the 14 GL-iNet routers are GLKVM-Cloud's, not UniFi's) — MongoDB's WiredTiger cache doesn't need much at this scale

Workload	Estimate	Basis
ATEM Overseer + Flock	5 GB	Unlike the other admin/control-plane containers below, these two are actually decoding video — up to 12 concurrent theatre streams apiece for their monitoring dashboards (see docs/bandwidth-analysis.md), not just pushing config or metadata
Restreamer, NDI Discovery, DERP, both ingest containers, GLKVM-Cloud (rttys+coturn), ATEM Fleet Admin, Tailscale router	9 GB	9 lightweight containers, ~1 GB each budgeted
Unraid OS + ZFS ARC (3 pools, ~9 TB usable combined)	8 GB	Soft ZFS guidance is roughly 1 GB RAM per TB of pool for decent ARC hit rate — this is a floor, not a hard requirement, ZFS is adaptive
Subtotal	~39 GB	Down from ~71 GB before the VMix instance VM (32 GB on its own) was removed from the design
Minimum, with headroom	64 GB	39 GB doesn't map to a clean multi-channel ECC DIMM configuration — 64 GB is the next practical capacity above it with real margin, not just rounding up to the subtotal
Recommended	96 GB	Gives ZFS ARC meaningfully more room across all three pools, and covers any future container additions without revisiting the DIMM population

ECC, not just capacity. All three storage pools above are ZFS — ZFS leans on RAM for checksumming and its ARC read cache, and a bit flip in ordinary (non-ECC) RAM can get silently written into a checksum or into the recording pool's parity/mirror data before anyone notices, which defeats the whole point of using ZFS for the one pool holding unrepeatable footage. ECC RAM catches and corrects that class of error before it propagates. Pair this with a platform that has *validated* ECC support (see the CPU section below) — ECC only protects against silent corruption if it's actually enabled and working, which isn't guaranteed on every board that merely accepts ECC modules electrically.

CPU

Workload	Estimate	Basis
BirdDog Central	2 cores	Routing/control, not encode
Docker stack (12 containers + the Tailscale subnet router + 3 DBs)	5-7 cores shared	Mostly I/O-bound; Restreamer's SRT relay (remux, not transcode, per this design's primary path) and ATEM Overseer + Flock (each decoding up to 12 concurrent monitoring streams) are the heaviest individual containers
Unraid OS/array overhead	2 cores	
Subtotal	9-11 cores	Down from 17-19 before the VMix instance VM (8 dedicated cores on its own) was removed from the design
Minimum	12 cores / 24 threads, high sustained (not just boost) clock	Rounds above the subtotal's high end — see platform-class discussion below, which matters more than squeezing out the last core or two

The platform class matters more than the core count, and this is worth spelling out rather than just naming a chip. A 16-core desktop CPU is easy to find — the actual constraint is everything *around* it:

- **PCIe lanes.** With up to 6 NVMe drives across the two NVMe pools above (2 in the container/VM mirror, 2-4 in the recording pool; the content pool is SATA), plus the 10GbE edit-LAN NIC (typically ×4 or ×8), this box still wants 20-30+ usable PCIe lanes — though the pressure has eased since the VMix VM (and with it a mandatory ×16 GPU slot) left the design. Mainstream consumer desktop platforms (the socket a typical Ryzen 9 or Core i9/Ultra 9 sits in) are now workable with careful

drive placement and bifurcation; a **workstation/HEDT-class platform** (Threadripper PRO or a high-end Xeon W) still clears it without any compromise, and remains the safer choice — but it's now a preference with a real mainstream alternative, no longer the only category that fits.

- **Validated ECC.** Some mainstream consumer boards technically accept ECC memory electrically, but whether ECC actually *functions* — gets detected, enabled, and does correction — varies board-to-board and isn't something to gamble on for the pool holding irreplaceable footage. Workstation platforms build ECC support into the platform itself, officially validated, not a maybe.
- **Sustained clock over peak boost.** The heavy loads here are continuous, not bursty — ATEM Overseer and Flock each decode up to 12 monitoring streams for the whole show, and the ingest pipelines run all day — so a CPU's *base* clock under sustained all-core load is a better predictor than its headline single-core boost number. Worth actually comparing base clocks across whichever specific chips are shortlisted, not just core count.

Trade-off worth being upfront about: workstation/HEDT platforms cost meaningfully more (CPU + motherboard together) than the mainstream desktop alternative, and idle power draw for 24/7 operation runs higher too. That's a real cost, not a rounding error. Since the VMix VM left the design, a high-end mainstream build with *validated* (not merely tolerated) ECC support is a legitimate cheaper alternative — the deciding check is the ECC validation and having enough lanes for the NVMe pools, not the platform label.

(IOMMU/VT-d support — previously a hard requirement here for GPU passthrough to the VMix VM — is no longer required by anything in the design. Virtually all current platforms have it anyway; it just no longer needs specific confirmation.)

GPU — no longer required

An earlier revision of this design carried a mothership VMix instance VM with a passthrough GPU, and this section specified a workstation-class NVENC card for it. **That VM has been removed** (docs/open-questions.md #10 — its role was never established; the theatre program feeds originate from the VMix node PCs, which have their own hardware). With it gone, nothing on this box needs a GPU:

- **Restreamer** relays SRT by remuxing, not transcoding — no encode.
- **BirdDog Central** is NDI routing/control — no encode.
- **ATEM Overseer and Flock** decode up to 12 monitoring/preview streams each, but these are 10 Mbps H.264 previews — CPU decode at this scale is already budgeted in the CPU table above.

The "decent" discrete GPU already in the box (docs/topology.md) can stay as spare capacity — potentially useful later for container-level decode/encode offload (e.g. if Overseer/Flock support hardware acceleration, or a future transcode workload appears) — but the spec no longer *requires* any GPU, and no replacement or upgrade should be budgeted for one.

Summary — minimum spec to check the existing server against

Component	Minimum	Why this class
CPU	12 cores / 24 threads, strong sustained clock — workstation/HEDT platform	PCIe lane count for up to 6 NVMe drives, validated ECC support; lane pressure eased since the VMix VM (and its ×16 GPU slot) left the design

Component	Minimum	Why this class
	preferred, high-end mainstream with validated ECC acceptable	
Motherboard	Validated ECC support, enough PCIe 4.0/5.0 lanes for the 2 NVMe pools (up to 6 drives) plus the 10GbE NIC	Same reasoning as CPU — the two are a package
RAM	64 GB minimum, 96 GB recommended, ECC, populate all available channels	ZFS data integrity across all 3 pools; subtotal ~39 GB since the VMix VM's 32 GB left the budget
GPU	None required — the on-hand GPU stays as spare capacity only	Nothing in the design encodes on this box any more; see the GPU section
Container/VM pool	500 GB usable, mirrored, consumer-grade DRAM-cached NVMe	Latency-sensitive VM/DB workload; consumer endurance is more than enough
Content pool	500 GB usable, mirrored, consumer-grade DRAM-cached SATA SSD	Gentle Nextcloud file I/O; no case for spending more
Recording pool	8 TB usable, mirrored (in practice 2× 7.68 TB ≈ 7.7 TB, or 2× 15.36 TB if 8 TB is a hard floor — see the drive-size note above), enterprise-grade NVMe with power-loss protection	PLP is the deciding factor — this pool is the authoritative archive (the edit-suite NAS dual-write copy may be a subset, and may be RAID 0 — see docs/live-editing.md); confirm event duration against the capacity table above first
Network	2× 1GbE, bonded LACP (already decided) — 2× 2.5GbE strongly preferred, required if laptop ISO inputs are pulled live fleet-wide — plus a separate edit-LAN-facing interface (10GbE) for the dual-write leg	2× 1GbE fits normal operation at the default live scope (camera + program) comfortably; the mass-NDI-fallback worst case fits in aggregate but depends on the LACP hash splitting ~14 WireGuard flows evenly (or on pausing ATEM ingest fleet-wide). 2× 2.5GbE removes that dependency. Either way, the edit-suite dual-write must never ride the theatre-facing bond, and the gateway's routing capacity and the mothership-side routing path (open-questions #17, #18) must be checked

Everything above is a calculated **target**, not a purchase order — the actual box is already on hand per docs/topology.md . Next step is checking its real spec against this table and updating docs/open-questions.md #1 with the result.